

A EMPRESA DOS DEPLOYS INFINITOS

ONDE OS MICROSERVIÇOS NUNCA DORMEM

Luis Zancanela



01

BEM-VINDO À EMPRESA DOS DEPLOYS INFINITOS

Bem-vindo à Empresa dos Deploys Infinitos

O CEO dos Containers e o RH dos Pods

Imagine o Kubernetes como um **sistema de gestão distribuído**: não há um único “CEO”, mas vários componentes que trabalham juntos para manter o ambiente no **estado desejado**. Quando a demanda cresce, o sistema **cria pods** e quando cai, **reduz réplicas**. Essas ações são automáticas e baseadas na especificação declarada.



02

O QUE TEM NO KUBERNETES?

Container e Pod

Os Colaboradores e as Salas

Container = Colaborador

O **container** é um processo isolado que executa uma função específica. Na empresa é o **colaborador** com sua mesa, computador e ferramentas, executando uma tarefa de forma independente.

Pod = Sala Modular

O **pod** agrupa containers com rede e volumes compartilhados. Pense numa **sala modular** com divisórias móveis, que pode ser criada, reconfigurada ou removida conforme a necessidade.

Um pod pode conter apenas um container, como a “sala do Fulano”, mas isso não torna container e pod a mesma coisa.



Workload, Node e Cluster

Departamentos, Andares e Prédio

Workload = Departamento

Workload refere-se a recursos como Deployment, StatefulSet, DaemonSet e Job. **Cada tipo de workload** representa um **departamento** que define regras e metas. Por exemplo, um Deployment é o departamento que decide quantas salas abrir e como manter o time. O workload garante que as salas modulares (pods) existam e sigam a especificação declarada.

Node = Andar

Node é a máquina física ou virtual que executa pods. Na empresa é o **andar** do prédio onde várias salas estão alocadas. Nodes podem falhar, ser drenados ou substituídos, por isso não confie em permanência.

Cluster = Prédio

Cluster é o conjunto de nodes mais o control plane que mantém tudo funcionando. Na empresa é o prédio inteiro com todos os andares e departamentos operando juntos.

HIERARQUIA DO KUBERNETES



Execução → Gerenciamento → Orquestração → Máquina → Infraestrutura

03

DEPLOYS INFINITOS EM MOVIMENTO

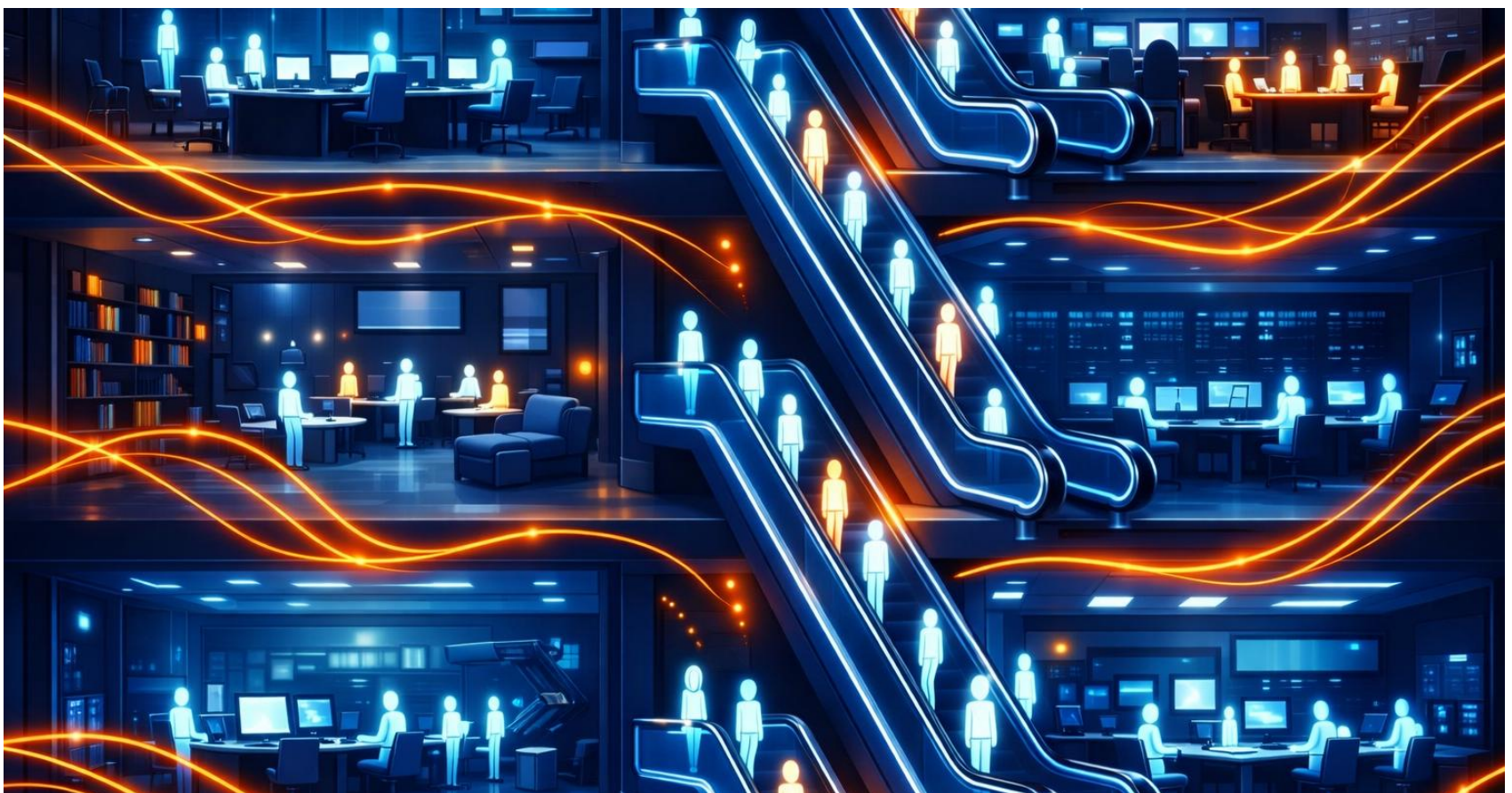
Deploys Infinitos em Movimento

Autoscaling, resiliência e gestão contínua

O Kubernetes atua como um gestor que monitora a empresa em tempo real e reconcilia o estado atual com o estado desejado. Ele observa métricas e eventos e age para manter cada departamento funcionando conforme o plano.

Quando a demanda de um departamento aumenta, o sistema abre novas salas modulares, ou seja, cria mais pods para atender a carga. Quando a demanda cai, ele reduz réplicas para economizar recursos.

A resiliência vem da reconciliação e do auto healing. Se uma sala falha, o Kubernetes reinicia ou reprovisiona a sala em outro andar. Se uma aplicação não responde, probes de readiness e liveness impedem que ela receba tráfego ou forçam reinício.



04

KUBERNETES NOS PROVEDORES DE NUVEM

Kubernetes nos Provedores de Nuvem

Filiais Globais da Empresa dos Deploys Infinitos

Na Deploys Infinitos, as filiais na nuvem seguem a mesma lógica de gestão. Dominar os conceitos (pods, workloads, nodes, cluster) permite trabalhar em qualquer filial sem recomeçar do zero. O que muda entre as filiais são as portas de entrada, os serviços auxiliares e os atalhos operacionais.

Azure Kubernetes Service (AKS)

Foca na integração com o ecossistema Azure, CI/CD com GitHub Actions e monitoramento via Azure Monitor.

Amazon Elastic Kubernetes Service (EKS)

Oferece alta escalabilidade e integração com serviços AWS como EC2, IAM e CloudWatch.

Google Kubernetes Engine (GKE)

Originado no Google e doado à comunidade, GKE oferece integração nativa com o Google Cloud e recursos como Autopilot e autoscaling inteligente.



05

QUANDO O DEPLOY DÁ ERRADO

Por que meu Pod não sobe?

Por Que Colaboradores Não Conseguem Trabalhar

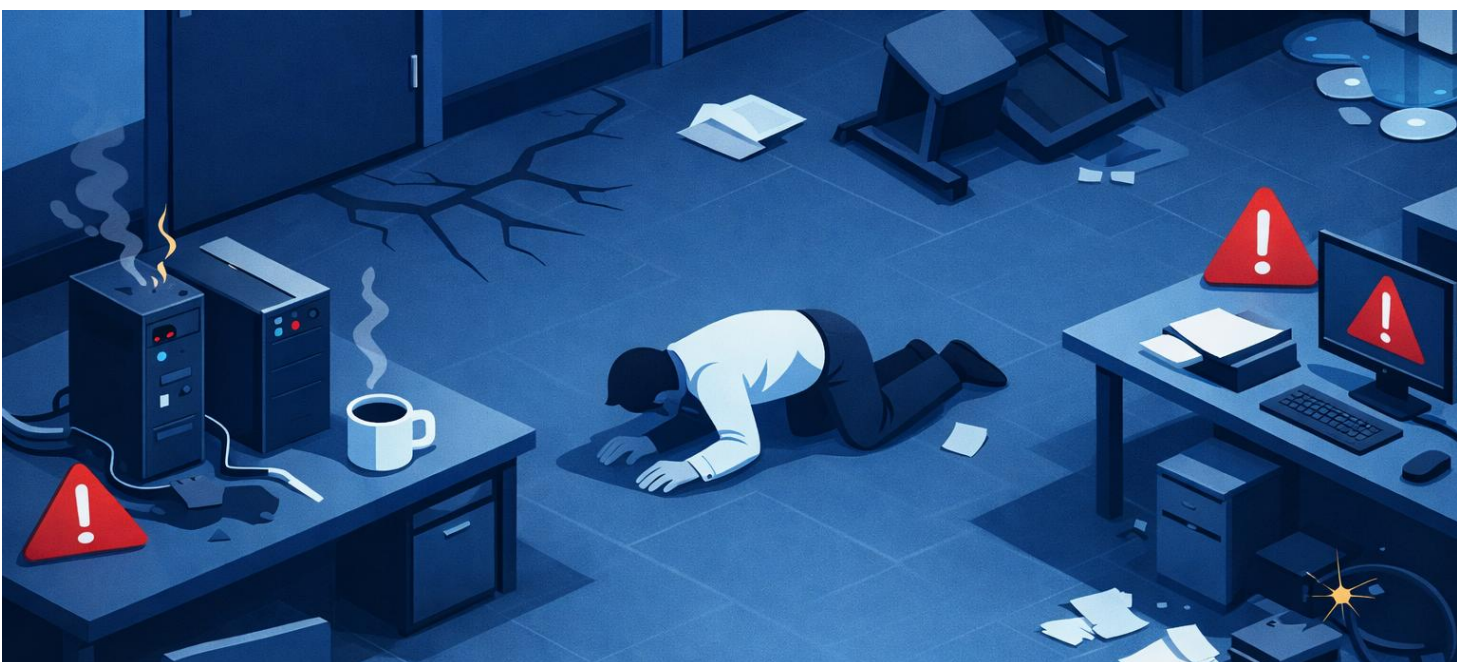
OOMKilled: “O colaborador desmaiou por excesso de trabalho.”
Como identificar: o pod reinicia repetidamente e no describe aparece Reason: OOMKilled. Significa que o container ultrapassou o limite de memória definido.

ImagePullBackOff / ErrImagePull: “Crachá inválido ou registro privado sem acesso.”

O Kubernetes não consegue baixar a imagem do container.
Verifique o nome da imagem, a tag e as credenciais do registry (imagePullSecrets).

CrashLoop por dependência externa: “O colaborador depende de outro setor que está fechado.”

O container inicia, tenta conectar a um banco de dados ou serviço externo que não está disponível, e falha. O pod fica em CrashLoopBackOff.



Fluxo de Diagnóstico

O Caminho Para Encontrar o Problema

1. **kubectl get pods** → Verificar o status. Procure por: CrashLoopBackOff, ImagePullBackOff, Pending, OOMKilled.
2. **kubectl describe pod <nome>** → Ler a seção Events. Procure por: FailedScheduling, Failed to pull image, Unhealthy, OOMKilled.
3. **kubectl logs <nome>** → Analisar os logs do container. Procure por: stack traces, connection refused, timeout, permission denied.
4. **kubectl exec -it <nome> -- /bin/sh** → Entrar na sala do colaborador para investigar diretamente. Verificar variáveis de ambiente, conectividade de rede, arquivos de configuração.
5. **Checar node e métricas** → **kubectl top nodes** / **kubectl top pods**. Verificar se há pressão de recursos no andar (node) ou se o prédio (cluster) está sobrecarregado.



Mensagens de Erro e Interpretação

Traduzindo o Que a Empresa Está Dizendo

Back-off restarting failed container

“O colaborador continua tropeçando ao entrar. O gestor está esperando antes de tentar novamente.” Ação: verificar logs para entender a causa da falha.

0/3 nodes are available: insufficient memory

“Todos os andares do prédio estão lotados, sem mesa para o novo colaborador.” Ação: liberar recursos ou adicionar mais nodes ao cluster.

Failed to pull image "app:v2": not found

“O crachá do colaborador aponta para uma foto que não existe no arquivo.” Ação: verificar nome e tag da imagem no registry.

Readiness probe failed: connection refused

“O colaborador foi chamado, mas ainda não está pronto para atender.” Ação: verificar se a aplicação está iniciando corretamente na porta configurada.

Liveness probe failed: HTTP probe failed with statuscode: 500

“A vistoria periódica descobriu que o colaborador está em estado crítico.” Ação: verificar logs da aplicação para erros internos.



Probes e Limites de Recursos

Verificando a Saúde e o Espaço dos Colaboradores

readinessProbe: “O colaborador está pronto para ser chamado?”

Verifica se o container já está apto a receber tráfego. Enquanto não passar, o Service não direciona requisições para ele.

livenessProbe: “O colaborador ainda está trabalhando?”

Verifica periodicamente se o container está funcionando. Se falhar, o Kubernetes reinicia o pod automaticamente.

Requests: “A sala tem espaço suficiente para receber trabalho?”

O mínimo de CPU e memória garantido para o container. O scheduler usa esse valor para decidir em qual node alocar o pod.

Limits — “Máximo que pode ocupar”

O teto de recursos. Se o container ultrapassar o limite de memória, será encerrado (OOMKilled).

```
resources:
  requests:          # tamanho mínimo da sala reservada
    memory: "128Mi"
    cpu: "250m"
  limits:           # máximo que pode ocupar
    memory: "256Mi"
    cpu: "500m"
readinessProbe:     # pronto para atender?
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 5
  timeoutSeconds: 2
livenessProbe:      # ainda está vivo?
  httpGet:
    path: /health
    port: 8080
  initialDelaySeconds: 20
  timeoutSeconds: 2
```

Playbook de Escalonamento

Quem Faz o Quê Quando Algo Dá Errado

Desenvolvedor (Júnior/Pleno):

- Executar o checklist básico: `kubectl get pods`, `describe`, `logs`
- Verificar configuração da aplicação (variáveis de ambiente, portas, imagem)
- Documentar o que foi encontrado antes de escalar

Desenvolvedor Sênior:

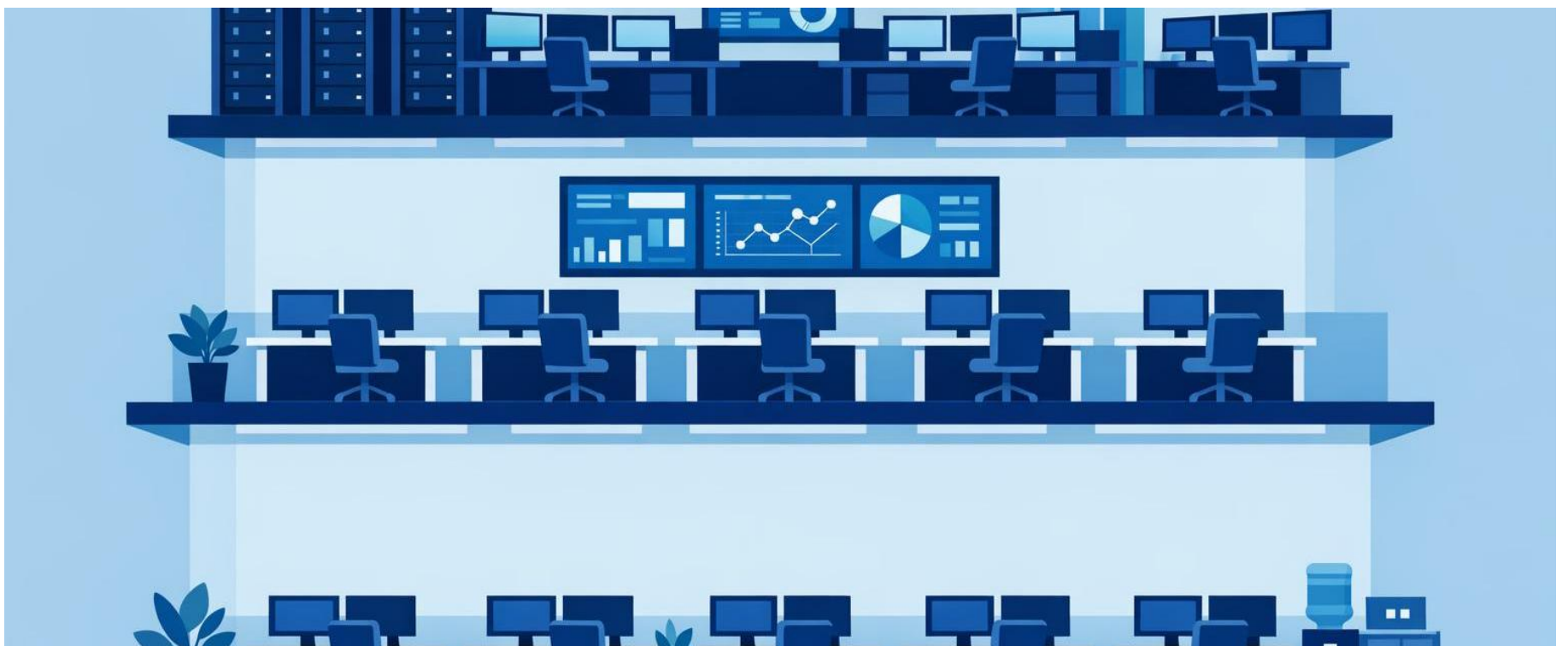
- Análise de causa raiz: revisar probes, limits, dependências externas
- Ajustar configurações de Deployment (replicas, resources, probes)
- Validar se o problema é da aplicação ou da infraestrutura

SRE / Infra:

- Investigar estado do node: `kubectl describe node`, `top nodes`
- Verificar quotas de namespace, RBAC e políticas de rede
- Registrar incidente e aplicar correções de infraestrutura

Quando abrir ticket para SRE:

- OOMKilled repetido após ajuste de limits
- Node sem recursos disponíveis (`SchedulingFailed`)
- Erro de imagem em registry privado (credenciais/rede)
- Falha de rede entre serviços (DNS, `NetworkPolicy`)



006

CONVERSANDO COM O GESTOR

Comandos Essenciais

O Kit de Ferramentas do Gestor da Empresa

kubectl get

Listar colaboradores, setores e turnos

“Quem está trabalhando agora?”

kubectl describe

Ficha completa do colaborador

“Me conta tudo sobre esse colaborador.”

kubectl logs

O que ele fazia antes do problema

“Mostra o diário de bordo dele.”

kubectl exec

Entrar na sala para investigar

“Deixa eu sentar na mesa dele um momento.”

kubectl apply

Enviar novas ordens para a empresa

“Implementa essas mudanças agora.”

kubectl delete

Encerrar um turno ou demitir

“Pode encerrar o expediente dele.”

kubectl get

Quem Está Trabalhando Agora?

COMANDO

```
$ kubectl get pods -o wide
```

OUTPUT

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
api-deploy-7d4b9-xk2f	1/1	Running	0	2h	10.244.1.5	node-01
api-deploy-7d4b9-m3nq	1/1	Running	0	2h	10.244.2.8	node-02
web-deploy-5c8a2-j9pl	0/1	Pending	0	5m	<none>	<none>

INTERPRETAÇÃO

Duas salas (api) estão ativas nos seus andares, com os colaboradores em operação. Uma terceira sala (web) está na fila de espera e ainda não encontrou espaço disponível no prédio. O status Pending indica que o gestor não conseguiu alocá-la em nenhum andar.

kubectl describe

A Ficha Completa da Sala de Trabalho

COMANDO

```
$ kubectl describe pod api-deploy-7d4b9-xk2f
```

OUTPUT

Events:

Type	Reason	Age	Message
----	-----	---	-----
Normal	Scheduled	2h	Successfully assigned to node-01
Normal	Pulled	2h	Container image "api:v1.2" pulled
Normal	Started	2h	Started container api

INTERPRETAÇÃO

A ficha da sala mostra que a sala foi criada com sucesso no 1º andar (node-01), recebeu o colaborador (imagem baixada) e iniciou o expediente normalmente. Se houvesse problemas, os Events mostrariam avisos (Warning) com detalhes.

kubectl logs

O Diário de Bordo do Colaborador

COMANDO

```
$ kubectl logs api-deploy-7d4b9-xk2f --tail=5
```

OUTPUT

```
2025-01-15T10:30:01Z INFO Server started on port 8080
2025-01-15T10:30:05Z INFO Connected to database
2025-01-15T10:35:12Z WARN Slow query detected: 2.3s
2025-01-15T10:40:00Z ERROR Connection refused: redis:6379
2025-01-15T10:40:01Z ERROR Health check failed
```

INTERPRETAÇÃO

O diário de bordo mostra que o colaborador começou bem, mas encontrou problemas: uma operação lenta e depois perda de contato com outro setor (Redis). Quando aparecem linhas ERROR, é sinal de que algo precisa de atenção imediata.

kubectl exec

Entrando na Sala do Colaborador

COMANDO

```
$ kubectl exec -it api-deploy-7d4b9-xk2f -- /bin/sh
```

OUTPUT

```
/ # curl localhost:8080/health
{"status":"degraded","db":"ok","cache":"unreachable"}
/ # env | grep DATABASE
DATABASE_URL=postgres://db-svc:5432/app
/ # nslookup redis-svc
Server: 10.96.0.10    Address: 10.96.0.10:53
** server can't find redis-svc: NXDOMAIN
```

INTERPRETAÇÃO

Entramos na sala do colaborador para investigar pessoalmente. Descobrimos que a saúde está degradada porque o setor de cache (Redis) não está acessível, visto que o DNS não encontra o serviço. Isso confirma o que os logs indicavam.

kubectl apply

Enviando Novas Ordens Para a Empresa

COMANDO

```
$ kubectl apply -f deployment.yaml
```

OUTPUT

```
deployment.apps/api-deploy configured  
service/api-svc unchanged
```

INTERPRETAÇÃO

As novas instruções foram enviadas e a empresa aceitou. O departamento (Deployment) foi atualizado com as mudanças. A portaria (Service) não precisou de alteração. O Kubernetes agora vai reorganizar os colaboradores conforme as novas regras.

kubectl delete

Encerrando o Expediente

COMANDO

```
$ kubectl delete pod api-deploy-7d4b9-xk2f
```

OUTPUT

```
pod "api-deploy-7d4b9-xk2f" deleted
```

INTERPRETAÇÃO

O expediente nessa sala foi encerrado. Como ele faz parte de um Deployment, o gestor (ReplicaSet) já está providenciando um substituto automaticamente. Se o delete fosse no Deployment inteiro, todos os colaboradores daquele departamento seriam dispensados. Use com cuidado!

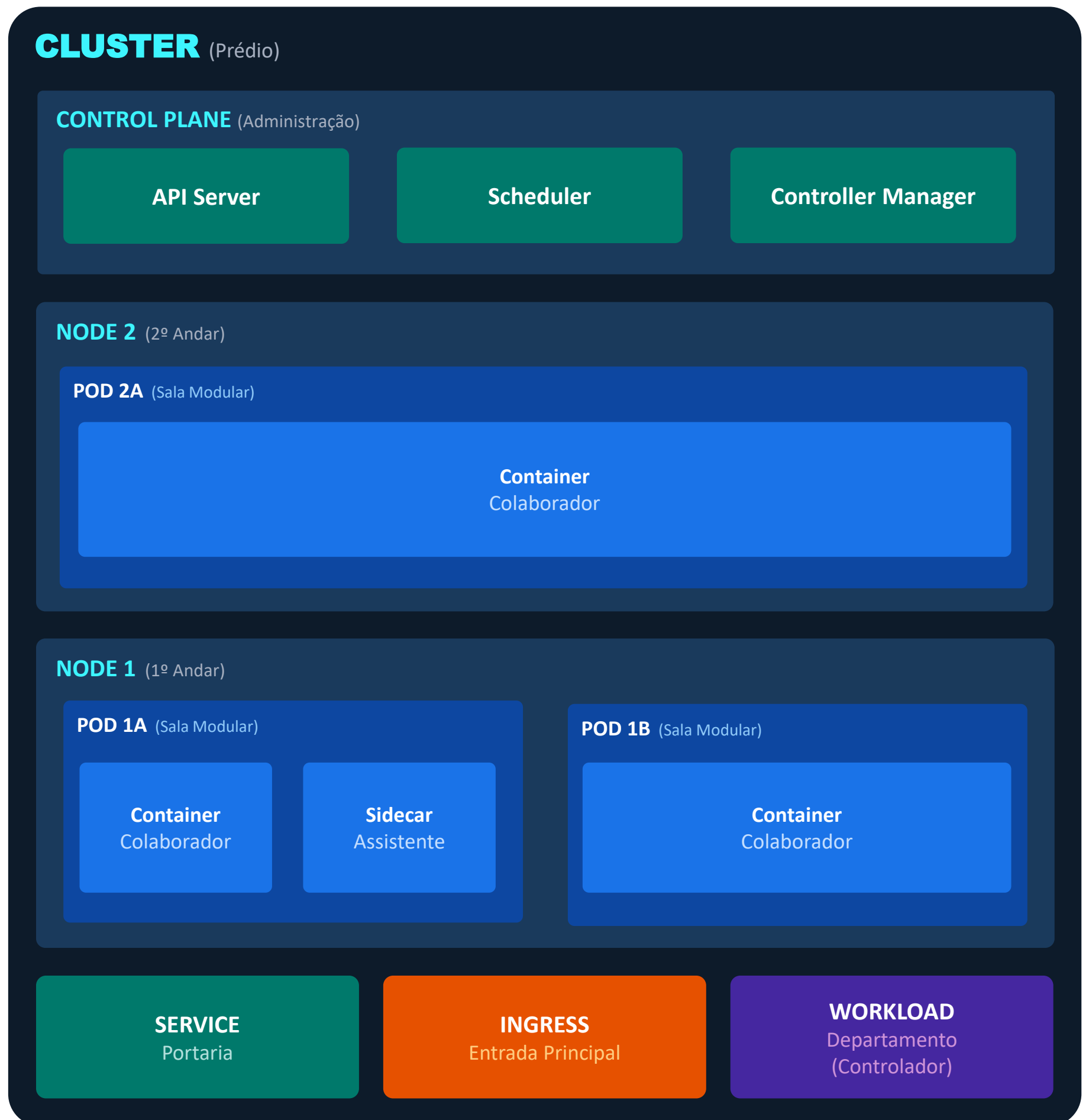
Cuidado: kubectl delete em um Deployment remove todos os pods. Sempre confirme o recurso antes de executar.

07

A PLANTA DA EMPRESA

Arquitetura Kubernetes

A Planta Completa da Empresa dos Deploys



Planta Técnica: YAML Comentado

O Blueprint da Empresa em Código

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deploy      # tipo de departamento
spec:
  replicas: 3           # abrir 3 salas para atender demanda
  selector:
    matchLabels:
      app: api          # etiqueta para identificar colaboradores
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api      # nome do colaborador
          image: api:v1.2 # crachá com foto e versão
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080 # porta de atendimento
      resources:
        requests:        # Tamanho mínimo da sala reservada
          memory: "128Mi"
          cpu: "250m"
        limits:          # máximo que pode ocupar
          memory: "256Mi"
          cpu: "500m"
      readinessProbe:    # pronto para atender?
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 5
        timeoutSeconds: 2
      livenessProbe:     # ainda está vivo?
        httpGet:
          path: /health
          port: 8080
        initialDelaySeconds: 20
        timeoutSeconds: 2
---
apiVersion: v1
kind: Service          # portaria do departamento
metadata:
  name: api-svc
spec:
  selector:
    app: api           # direcionar para colaboradores com esta etiqueta
  ports:
    - port: 80         # porta da portaria
      targetPort: 8080 # porta real da sala do colaborador
  type: ClusterIP      # acesso interno ao prédio
```

08

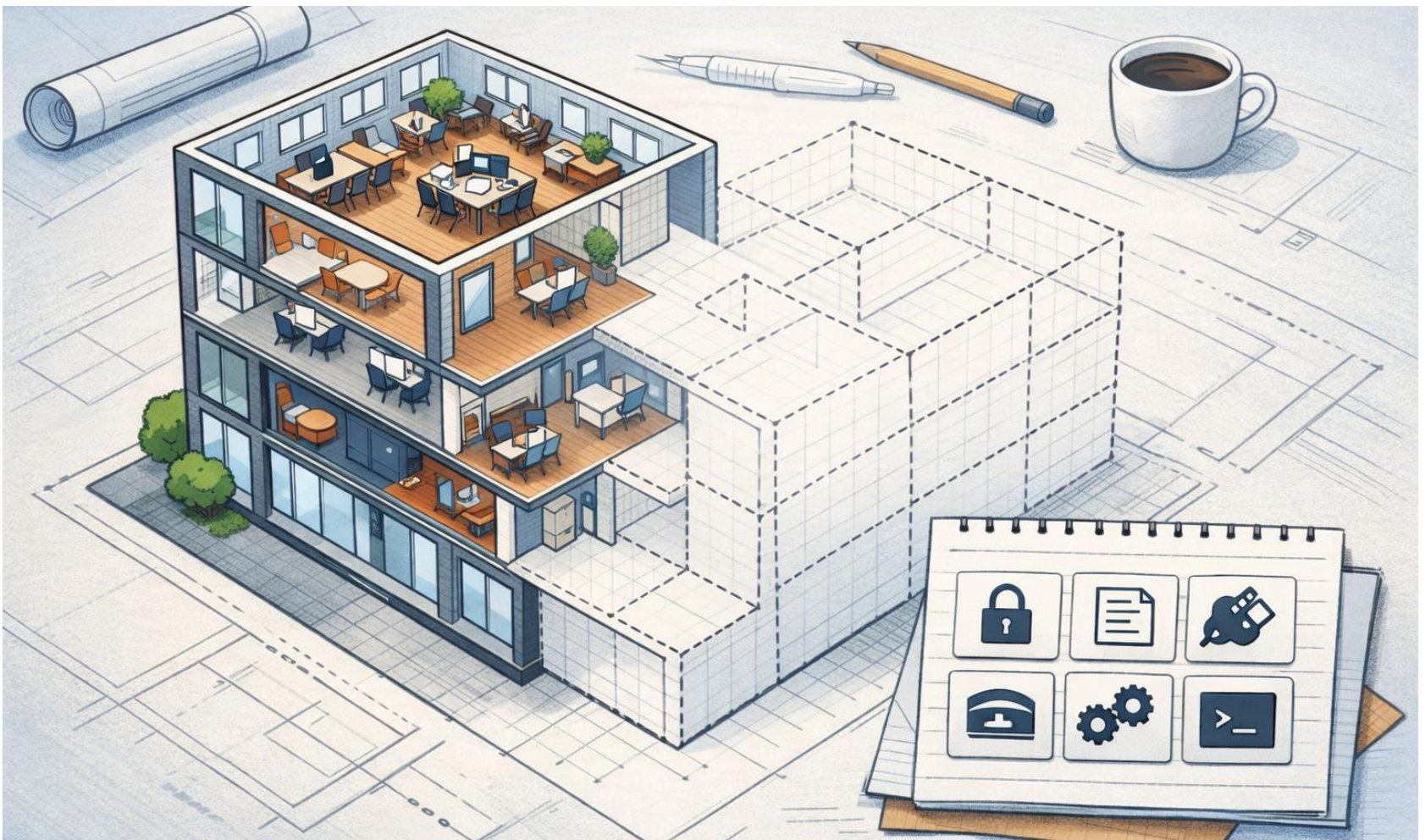
**ESTE É
APENAS O
COMEÇO**

Este é Apenas o Começo

A Deploys Infinitos ainda está no papel

Você conheceu a estrutura da nossa empresa imaginária: salas, andares, departamentos e colaboradores. Mas toda empresa real precisa de muito mais para funcionar. Ela precisa de portas de entrada, que representam o **Ingress**. Precisa de ramais internos, que fazem o papel dos **Services**. Precisa de documentos e políticas, que são os **ConfigMaps e Secrets**. Precisa de arquivos guardados, que são os **Volumes**. E também precisa da diretoria que coordena tudo, formada pelo **Control Plane, Scheduler e Kubelet**.

Essas partes ainda não apareceram na história, e é exatamente por isso que esta jornada está só começando.



**OBRIGADO
POR LER!**

Obrigado por Ler!

Aguardo seu feedback!

O que achou da analogia do Kubernetes como uma Empresa?

Sobre o Autor

Olá, meu nome é Luis Zancanela e sou apaixonado por tecnologia e desenvolvimento de sistemas. Atuo na área de TI há mais de 19 anos, sendo mais de 4 anos como desenvolvedor backend Java, inclusive trabalhando com aplicações em Kubernetes. Ter o conhecimento da infraestrutura onde a aplicação está executando é muito importante principalmente quando tem algum “incêndio” em produção.



Abaixo está meu perfil no LinkedIn para nos conectarmos:



<https://www.linkedin.com/in/luis-zancanela/>

Esse ebook foi gerado com base em conteúdo textual e visual gerado por IA e revisado por humano.

Criado com paixão, dedicação e o compromisso sincero de compartilhar conhecimento 🧑💻 ❤️ 🤝 📚